

File Handling in C++

Reading and Writing Files Using Streams

Le Nhut Nam

Department of Computer Science

December 17, 2025

Outline

- 1 Introduction to File Handling
- 2 Text File Operations
- 3 Binary File Operations
- 4 File Positioning
- 5 Advanced Topics
- 6 Best Practices and Summary

Why File Handling?

- Variables store data only during program execution
- Files allow data to be stored permanently
- Useful for:
 - Saving program output
 - Processing large datasets
 - Logging and configuration files
 - Data exchange between programs

File Streams in C++

C++ uses **stream-based I/O** for file handling.

Header File

```
#include <fstream>
```

File Stream Classes

- `ifstream` — input file stream (read)
- `ofstream` — output file stream (write)
- `fstream` — input and output

Basic Steps of File Handling

- 1 Declare a file stream object
- 2 Open the file
- 3 Perform read or write operations
- 4 Close the file

Important

Always close files to:

- Release system resources
- Ensure data is written correctly

Writing to a File (ofstream)

```
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("data.txt");
    outFile << "Hello, File Handling in C++!" << endl;
    outFile << "This is a second line." << endl;
    outFile.close();
    return 0;
}
```

- Creates data.txt if it does not exist
- Overwrites file if it already exists

Reading from a File (ifstream)

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream inFile("data.txt");
    string line;

    while (getline(inFile, line)) { // getline() reads one line at a
        time
        cout << line << endl;
    } // Stops automatically at end-of-file

    inFile.close();
    return 0;
}
```

Checking File Open Status

```
ifstream inFile("data.txt");  
  
if (!inFile.is_open()) {  
    cout << "Error: Cannot open file!" << endl;  
    return 1;  
}
```

Why Check?

- File may not exist
- Permission may be denied
- Prevents runtime errors

File Opening Modes

Mode	Description
<code>ios::in</code>	Read from file
<code>ios::out</code>	Write to file
<code>ios::app</code>	Append to file
<code>ios::trunc</code>	Erase contents
<code>ios::binary</code>	Binary file mode
<code>ios::ate</code>	Start at end of file

```
ofstream file("log.txt", ios::app);  
// Combines modes with |  
fstream f("data.txt", ios::in | ios::out);
```

Reading Multiple Values

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream inFile("numbers.txt");
    int x;

    while (inFile >> x) { // Operator >> reads formatted data
        cout << x << " ";
    } // Loop stops at EOF automatically

    inFile.close();
    return 0;
}
```

Reading Mixed Data Types

```
// File: student.txt contains: John 85.5 A

ifstream inFile("student.txt");
string name;
double score;
char grade;

inFile >> name >> score >> grade;

cout << name << " scored " << score
      << " with grade " << grade << endl;
```

Note

The >> operator automatically skips whitespace between values.

Text Files vs Binary Files

Text Files

- Human-readable
- Portable across systems
- Larger file size
- Slower I/O operations

Binary Files

- Faster I/O
- Smaller file size
- Not human-readable
- Platform-dependent

Writing Binary Files

```
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("data.bin", ios::binary);

    int numbers[] = {10, 20, 30, 40, 50};

    outFile.write(reinterpret_cast<char*>(numbers),
                  sizeof(numbers));

    outFile.close();
    return 0;
}
```

- write() writes raw bytes to file
- reinterpret_cast converts pointer types

Reading Binary Files

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream inFile("data.bin", ios::binary);

    int numbers[5];

    inFile.read(reinterpret_cast<char*>(numbers), sizeof(numbers));

    for (int i = 0; i < 5; i++)
        cout << numbers[i] << " ";

    inFile.close();
    return 0;
}
```

Working with Structs in Binary Files

```
struct Student {  
    char name[50];  
    int age;  
    double gpa;  
};  
  
// Writing  
Student s = {"Alice", 20, 3.85};  
ofstream out("students.bin", ios::binary);  
out.write(reinterpret_cast<char*>(&s), sizeof(s));  
out.close();  
  
// Reading  
Student s2;  
ifstream in("students.bin", ios::binary);  
in.read(reinterpret_cast<char*>(&s2), sizeof(s2));  
in.close();
```

File Position Pointers

Each file stream maintains position pointers:

Position Pointers

- `get pointer` — current read position
- `put pointer` — current write position

Functions

- `tellg()` / `tellp()` — get current position
- `seekg()` / `seekp()` — set position

Seeking in Files

Direction	Description
<code>ios::beg</code>	Beginning of file
<code>ios::cur</code>	Current position
<code>ios::end</code>	End of file

```
ifstream file("data.txt");  
  
file.seekg(0, ios::end);    // Go to end  
int size = file.tellg();    // Get file size  
  
file.seekg(0, ios::beg);    // Go back to start  
file.seekg(10, ios::cur);    // Move 10 bytes forward
```

Getting File Size

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream file("data.txt", ios::ate);

    if (file.is_open()) {
        streampos size = file.tellg();
        cout << "File size: " << size << " bytes" << endl;
        file.close();
    }

    return 0;
}
```

Note

Using fstream for Read/Write

```
#include <fstream>
using namespace std;
int main() {
    fstream file("data.txt", ios::in | ios::out | ios::trunc);

    // Write data
    file << "Hello World" << endl;

    // Go back to beginning
    file.seekg(0, ios::beg);

    // Read data
    string line;
    getline(file, line);

    file.close();
    return 0;
}
```

File State Flags

Function	Description
good()	No errors occurred
eof()	End-of-file reached
fail()	Logical error on I/O
bad()	Read/write error
clear()	Reset error flags

```
ifstream file("data.txt");
while (file.good()) {
    // Perform operations
}
if (file.eof()) cout << "End of file reached";
```

RAII: Automatic Resource Management

```
void processFile(const string& filename) {  
    ifstream file(filename); // Opens file  
  
    if (!file) {  
        throw runtime_error("Cannot open file");  
    }  
  
    string line;  
    while (getline(file, line)) {  
        // Process line  
    }  
  
    // file.close() called automatically  
    // when file goes out of scope  
}
```

RAII: Automatic Resource Management (Cont)

RAII Principle

Destructor automatically closes file — no explicit `close()` needed.

Bonus: String Streams

```
#include <sstream>
#include <string>
using namespace std;
int main() {
    // Parse string into variables
    string data = "Alice 25 3.85";
    istringstream iss(data);

    string name;
    int age;
    double gpa;

    iss >> name >> age >> gpa;
```

Bonus: String Streams (Cont)

```
// Build formatted string
ostringstream oss;
oss << name << " is " << age << " years old";
string result = oss.str();

return 0
}
```


Common Mistakes

- Forgetting to close files
- Incorrect file path (relative vs absolute)
- Using wrong open mode
- Reading without checking file state
- Not handling errors properly
- Mixing text and binary operations

Best Practice

Always check file status and close files explicitly (or use RAII).

Best Practices

- ① Always check if file opened successfully
- ② Use RAII — let destructors handle cleanup
- ③ Prefer `getline()` for reading lines
- ④ Use binary mode for non-text data
- ⑤ Handle errors gracefully
- ⑥ Use meaningful file extensions

Tip

Use `std::filesystem` (C++17) for advanced file operations like checking existence, creating directories, etc.

Summary

- File handling uses `<fstream>` header
- Three main classes: `ifstream`, `ofstream`, `fstream`
- Text files: use `<<`, `>>`, `getline()`
- Binary files: use `read()`, `write()`
- File positioning: `seekg()`, `seekp()`, `tellg()`, `tellp()`
- Always validate file operations

Practice Exercise 1

Task: Student Records

- 1 Create a file `students.txt`
- 2 Write 5 student records (name and score)
- 3 Read the file and compute average score
- 4 Display student with highest score

Hint

Use `ofstream` for writing and `ifstream` for reading.

Practice Exercise 2

Task: Binary Database

- 1 Define a Product struct with: id, name, price
- 2 Write 3 products to a binary file
- 3 Read all products and display them
- 4 Update the price of product with id = 2

Hint

Use `seekp()` to position before writing updates.

- C++ Reference: <https://cplusplus.com/reference/fstream/>
- cppreference: <https://en.cppreference.com/w/cpp/io>
- Stroustrup, B. (2013). *The C++ Programming Language*

Thank You!

Questions?